

TECHNICAL SPECIFICATION V1.0

Project Manticore

Complete Technical Blueprint — Architecture, APIs, SDKs,
Code Examples & Integration Guides

Platform: ManticoreAI ShieldProbe

Modules: Assess · Defend · Fix · Continuous

Compiled: June 15, 2026

TECHNICAL REFERENCE

Table of Contents

1. System Architecture Blueprint

2. REST API Reference

2.1 Authentication

2.2 Engagements API

2.3 Findings API

2.4 GCE Management API

2.5 Reports API

2.6 Webhooks API

3. GCE Framework Deep-Dive

3.1 GCE Lifecycle

3.2 Six Mitigation Strategies

3.3 GCE Code Examples

4. SDK Integration Guide

4.1 Node.js SDK

4.2 Python SDK

4.3 Go SDK

4.4 Java SDK

5. ShieldProbe Assess Specification

5.1 Engagement Flow

5.2 Attack Surface Coverage

5.3 Evidence Format

6. ShieldProbe Defend Specification

6.1 Deployment Architecture

6.2 Runtime Behavior

7. ShieldProbe Fix Specification

8. ShieldProbe Continuous — CI/CD

9. Database Schema

10. Authentication & Security

11. Webhook/Event Reference

12. Working Integration Examples

1. System Architecture Blueprint

ManticoreAI ShieldProbe is built on a **layered microservices architecture** orchestrated by Kubernetes. The platform follows a single-tenant reasoning engine model where each customer engagement receives dedicated inference resources. All four modules — Assess, Defend, Fix, and Continuous — share the same core reasoning engine but expose distinct service interfaces.

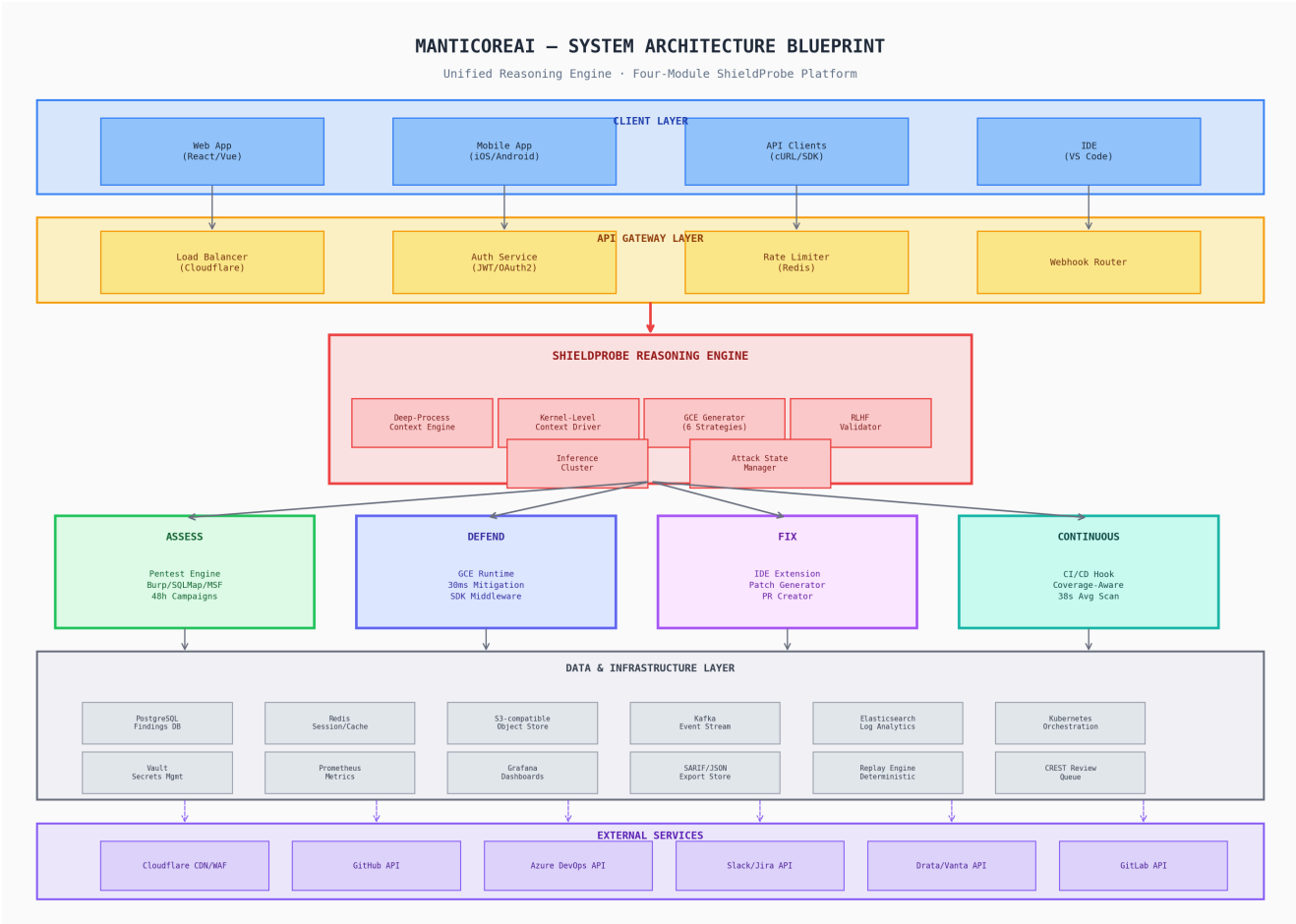


Figure 1 ManticoreAI ShieldProbe — Layered System Architecture Blueprint

Architecture Layers

Table 1 Architecture Layer Specifications

Layer	Components	Technology	Purpose
Client Layer	Web App, Mobile, API Clients, VS Code IDE	React, Swift, Kotlin, Express	User-facing interfaces
API Gateway	Load Balancer, Auth Service, Rate Limiter, Webhook Router	Cloudflare, Redis, Node.js	Traffic management, JWT/OAuth2 validation

Reasoning Engine	Deep-Process Context Engine, Kernel Driver, GCE Generator, RLHF Validator, Inference Cluster, Attack State Manager	Custom C/Rust kernels, Python inference, Kubernetes	Core AI/ML processing
Module Services	Assess, Defend, Fix, Continuous	Go microservices, Python agents	Domain-specific business logic
Data Layer	PostgreSQL, Redis, S3, Kafka, Elasticsearch	Managed cloud services	Persistent storage, caching, events
Infrastructure	Vault, Prometheus, Grafana, Replay Engine, CREST Queue	HashiCorp, CNCF stack	Security, observability, compliance
External Services	Cloudflare, GitHub, Azure DevOps, Slack, Jira, Drata	REST API integrations	Third-party ecosystem

Service Mesh Communication

Inter-service communication uses **mTLS-encrypted gRPC** for internal traffic and **REST over HTTPS** for external-facing APIs. The reasoning engine communicates with module services via a custom event bus backed by Kafka. All sensitive operations (GCE deployment, patch generation) require dual authorization: service account validation + user JWT.

Key Architectural Principle

Code stays on our infra. The reasoning engine processes all attack logic, vulnerability evidence, and patch generation on ManticoreAI-controlled infrastructure. Customer source code is never transmitted to OpenAI, Anthropic, or any third-party LLM provider. This is enforced at the network level by egress filtering on the inference cluster.

2. REST API Reference

The ManticoreAI API is organized around RESTful resources. All endpoints use HTTPS, accept/return JSON, and are versioned via URL path (/api/v1/). Base URL: `https://api.manticore.ai`

2.1 Authentication

API authentication uses **Bearer tokens** via JWT. Tokens are obtained through the OAuth2 flow or generated in the platform dashboard.

```
Authorization: Bearer <jwt_token>
Content-Type: application/json
X-API-Version: 2024-06-01
```

Table 2 Authentication Endpoints

Method	Endpoint	Description
POST	/auth/token	Obtain access token (OAuth2 client_credentials)
POST	/auth/refresh	Refresh expiring token
POST	/auth/revoke	Revoke active token
GET	/auth/me	Get current user/organization context

Token Request

```
POST /auth/token HTTP/1.1
Host: api.manticore.ai
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=YOUR_CLIENT_ID
&client_secret=YOUR_CLIENT_SECRET
&scope=assess:read assess:write defend:manage
```

Token Response

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIs... ",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "assess:read assess:write defend:manage",
  "org_id": "org_2vX9kL3mP7qR"
}
```

2.2 Engagements API

Table 3 Engagements Endpoints

Method	Endpoint	Description
GET	/api/v1/engagements	List all engagements
POST	/api/v1/engagements	Create new engagement
GET	/api/v1/engagements/{id}	Get engagement details
PATCH	/api/v1/engagements/{id}	Update engagement
DELETE	/api/v1/engagements/{id}	Cancel engagement
POST	/api/v1/engagements/{id}/launch	Launch engagement
GET	/api/v1/engagements/{id}/status	Get real-time status

Create Engagement

POST /api/v1/engagements

Create a new penetration testing engagement

Request Body:

```
{
  "name": "Q3-2025 Production Pentest",
  "targets": [
    {
      "type": "web_app",
      "url": "https://api.acme.com",
      "credentials": {
        "username": "pentest@acme.com",
        "password": "{{vault:secret_id}}"
      }
    },
    {
      "type": "api",
      "url": "https://api.acme.com/v2",
      "spec_url": "https://api.acme.com/openapi.json"
    }
  ],
  "methodology": "wstg_full",
  "testing_type": "greybox",
  "compliance_frameworks": ["soc2", "pci_dss"],
  "schedule": {
    "start_at": "2025-07-01T00:00:00Z",
    "timezone": "America/New_York"
  },
  "notifications": {
    "webhook_url": "https://acme.com/webhooks/manticore",
    "events": ["finding.discovered", "engagement.completed"]
  }
}
```

Response: 201 Created

```
{
  "id": "eng_9Kx2mP4vL7qR",
  "name": "Q3-2025 Production Pentest",
  "status": "scheduled",
  "targets": [...],
  "estimated_completion": "2025-07-03T00:00:00Z",
  "created_at": "2025-06-15T14:32:00Z",
  "dashboard_url": "https://app.manticore.ai/engagements/eng_9Kx2mP4vL7qR"
}
```

Get Engagement Status

GET

/api/v1/engagements/{id}/status

Response:

```
{
  "engagement_id": "eng_9Kx2mP4vL7qR",
  "status": "running",
  "phase": "exploitation",
  "progress_percent": 62,
  "findings_count": 7,
  "critical_count": 1,
  "high_count": 3,
  "medium_count": 2,
  "low_count": 1,
  "time_elapsed": "18h 32m",
  "estimated_remaining": "17h 28m",
  "current_activity": "Testing authentication flows on /v2/auth/refresh",
  "agent_state": {
    "active_sessions": 3,
    "footholds": 2,
    "pending_hypotheses": 5
  }
}
```

2.3 Findings API

Table 4 Findings Endpoints

Method	Endpoint	Description
GET	/api/v1/findings	List findings (paginated, filterable)
GET	/api/v1/findings/{id}	Get finding with full evidence
GET	/api/v1/findings/{id}/replay	Get deterministic replay script
GET	/api/v1/findings/{id}/evidence	Download evidence archive
PATCH	/api/v1/findings/{id}	Update finding status

Get Finding Detail

GET

/api/v1/findings/{id}

Response:

```
{
  "id": "fnd_3Xk9mP2vL5qR",
  "engagement_id": "eng_9Kx2mP4vL7qR",
  "severity": "critical",
  "category": "business_logic",
  "title": "Negative-amount withdrawal balance inflation",
  "description": "The /api/withdraw endpoint accepts negative amounts...",
  "cwe_id": "CWE-841",
  "cvss_score": 8.7,
  "status": "verified",
  "verified_by": "crest_consultant_7",
  "evidence": {
    "request": "POST /api/withdraw HTTP/1.1\nHost: api.acme.com\n...\n{\"account\": \"ACC-1234\", \"amount\": -50000}",
    "response": "HTTP/1.1 200 OK\n{\"status\": \"success\", \"new_balance\": 150000}",
    "screenshot_url": "https://cdn.manticore.ai/evidence/fnd_3Xk9...png",
    "replay_script": "https://api.manticore.ai/v1/findings/fnd_3Xk9.../replay"
  },
  "attack_chain": [
    { "step": 1, "action": "Identified withdraw endpoint via API recon" },
    { "step": 2, "action": "Tested negative amount input" },
    { "step": 3, "action": "Confirmed balance increased by abs(amount)" }
  ],
  "affected_routes": ["/api/withdraw", "/api/transfer", "/api/refund"],
  "gce_id": "gce_8Mk2pL5vX9qR",
  "created_at": "2025-07-01T18:42:00Z",
  "verified_at": "2025-07-02T08:15:00Z"
}
```

2.4 GCE Management API

Table 5 GCE Management Endpoints

Method	Endpoint	Description
GET	/api/v1/gces	List all GCEs for organization
GET	/api/v1/gces/{id}	Get GCE details and code
POST	/api/v1/gces/{id}/enable	Enable GCE on target
POST	/api/v1/gces/{id}/disable	Disable GCE
POST	/api/v1/gces/{id}/rollback	Rollback to previous version
GET	/api/v1/gces/{id}/metrics	Get runtime metrics

Get GCE Detail

GET

/api/v1/gces/{id}

Response:

```
{
  "id": "gce_8Mk2pL5vX9qR",
  "finding_id": "fnd_3Xk9mP2vL5qR",
  "strategy": "sanitize",
  "language": "javascript",
  "route_pattern": "POST /api/withdraw",
  "status": "active",
  "version": 3,
  "code": "defend.intercept('POST /api/withdraw', (req) => {\n  if (req.body.amount < 0) {\n    req.body.amount = Math.abs(req.body.amount);\n  }\n  return req;\n});",
  "latency_ms": 27,
  "invocations": 15420,
  "threats_blocked": 3,
  "deployed_at": "2025-07-02T10:00:00Z",
  "versions": [
    { "version": 1, "status": "deprecated", "deployed_at": "2025-07-02T09:30:00Z" },
    { "version": 2, "status": "deprecated", "deployed_at": "2025-07-02T09:45:00Z" },
    { "version": 3, "status": "active", "deployed_at": "2025-07-02T10:00:00Z" }
  ]
}
```

2.5 Reports API

Table 6 Reports Endpoints

Method	Endpoint	Description
GET	/api/v1/reports	List generated reports
POST	/api/v1/reports	Generate new report
GET	/api/v1/reports/{id}	Get report metadata
GET	/api/v1/reports/{id}/download	Download report file
GET	/api/v1/reports/{id}/findings	Get findings in report

Generate Report

POST /api/v1/reports

Request:

```

{
  "engagement_id": "eng_9Kx2mP4vL7qR",
  "format": "pdf",
  "compliance_framework": "pci_dss_4_1",
  "template": "audit_grade",
  "include_evidence": true,
  "include_remediation": true,
  "language": "en"
}

```

Response: 202 Accepted

```

{
  "id": "rpt_5Mk8pL2vX9qR",
  "status": "generating",
  "estimated_ready": "2025-07-03T00:15:00Z",
  "crest_certified": true,
  "signed_by": "John Smith (CREST CRT)"
}

```

2.6 Webhooks API

Table 7 Webhooks Endpoints

Method	Endpoint	Description
GET	/api/v1/webhooks	List configured webhooks
POST	/api/v1/webhooks	Create webhook
DELETE	/api/v1/webhooks/{id}	Delete webhook
POST	/api/v1/webhooks/{id}/test	Send test event

Webhook Event Types

Table 8 Supported Webhook Events

Event	Description	Payload Size
engagement.launched	Engagement has started	~2KB
engagement.completed	Engagement finished	~5KB
finding.discovered	New vulnerability found	~15KB (with evidence)
finding.verified	Finding passed CREST validation	~10KB
gce.deployed	GCE deployed to production	~3KB

gce.triggered	GCE blocked a request	~2KB
report.generated	Report ready for download	~1KB
ci_run.completed	Continuous scan finished	~8KB

3. GCE Framework Deep-Dive

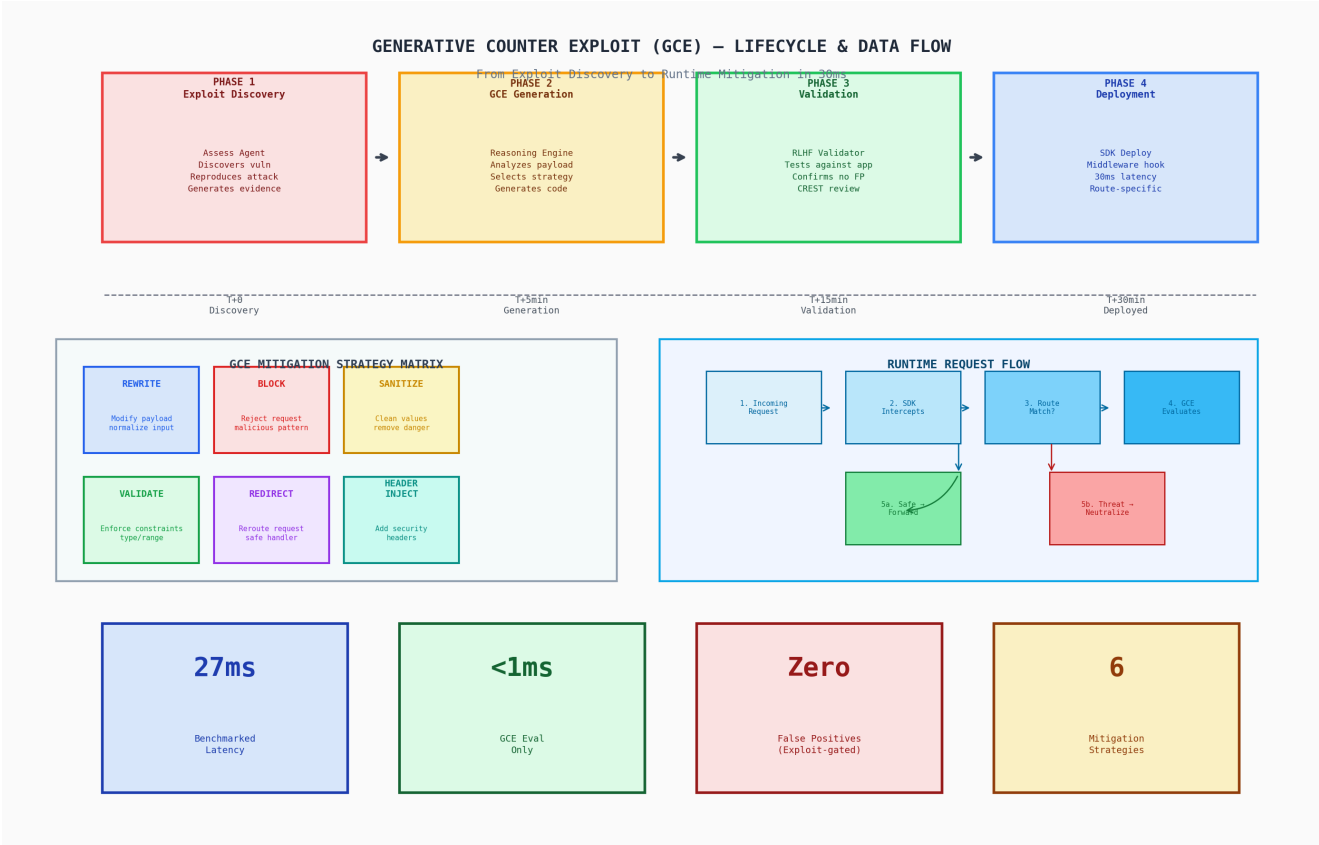


Figure 2 Generative Counter Exploit — Four-Phase Lifecycle and Runtime Flow

3.1 GCE Lifecycle

The GCE lifecycle consists of four distinct phases, moving from exploit discovery to runtime mitigation:

Table 9 GCE Lifecycle Phases

Phase	Duration	Actor	Output
Phase 1: Exploit Discovery	Variable engagement)	(during Assess AI Agent	Confirmed finding with reproducible evidence
Phase 2: GCE Generation	~5 minutes	Reasoning Engine	Generated code, selected strategy, route pattern
Phase 3: Validation	~10 minutes	RLHF Validator + CREST	Validated, versioned GCE ready for deployment
Phase 4: Deployment	~30 seconds	Defend SDK	Active middleware interceptor

3.2 Six Mitigation Strategies

Each GCE employs exactly one of six mitigation strategies, selected by the reasoning engine based on vulnerability class, payload structure, and application context.

Table 10 Mitigation Strategy Selection Matrix

Strategy	Mechanism	Best For			Latency Impact
Rewrite	Modifies request payload before it reaches the handler	Input coercion	normalization, type		~5ms
Block	Terminates request with error response	Confirmed malicious patterns			~2ms
Sanitize	Cleans dangerous values while preserving request flow	Negative injection	numbers, HTML/JS		~8ms
Validate	Enforces schema constraints (type, range, format)	Schema violations, conditions	boundary		~3ms
Redirect	Reroutes to safe fallback handler	Deprecated endpoints, known-bad paths			~4ms
Header Inject	Adds security headers to response	CSP, HSTS, X-Frame-Options gaps			~1ms

3.3 GCE Code Examples

Strategy: Sanitize (Negative Amount Withdrawal)

```
// GCE-2847 · SANITIZE strategy · Banking API
// Vulnerability: Negative amount withdrawal inflates balance
// Route: POST /api/withdraw
// Benchmarked latency: 27ms total, <1ms GCE eval

defend.intercept('POST /api/withdraw', (req) => {
  // Sanitize: convert negative amounts to absolute values
  if (req.body.amount !== undefined && req.body.amount < 0) {
    req.body.amount = Math.abs(req.body.amount);
    defend.log('GCE-2847', 'sanitized_negative_amount', {
      original: req.body.amount * -1,
      normalized: req.body.amount,
      account: req.body.account,
      timestamp: Date.now()
    });
  }
  return req;
});
```

Strategy: Validate (Schema Enforcement)

```
// GCE-1923 · VALIDATE strategy · User Registration API
// Vulnerability: Mass assignment via extra fields in user object
// Route: POST /api/v1/users

defend.intercept('POST /api/v1/users', (req) => {
  const allowedFields = ['email', 'password', 'first_name', 'last_name'];
  const incomingFields = Object.keys(req.body);

  const extraFields = incomingFields.filter(f => !allowedFields.includes(f));
  if (extraFields.length > 0) {
    defend.log('GCE-1923', 'mass_assignment_blocked', {
      extraFields,
      ip: req.ip,
      timestamp: Date.now()
    });
    // Block: remove unauthorized fields
    extraFields.forEach(f => delete req.body[f]);
  }

  // Validate email format
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (req.body.email && !emailRegex.test(req.body.email)) {
    return defend.block(400, { error: 'Invalid email format' });
  }

  return req;
});
```

Strategy: Block (SQL Injection Pattern)

```
// GCE-4451 · BLOCK strategy · Search API
// Vulnerability: SQL injection in search parameter
// Route: GET /api/search?q=<payload>

defend.intercept('GET /api/search', (req) => {
  const q = req.query.q || '';

  // Block known SQL injection patterns
  const sqlPatterns = [
    /(\%27)|(\')|(\-\-)|(\%23)|(\#)/i,
    /((\%3D)|(\=))^[^n]*(\%27)|(\')|(\-\-)|(\%3B)|(;)/i,
    /\w*(\%27)|(\')((\%6F)|o|(\%4F))((\%72)|r|(\%52))/i,
    /((\%27)|(\'))union/i,
    /exec(\s|\+)+(s|x)p\w+/i,
    /UNION\s+SELECT/i,
    /;\s*DROP\s+TABLE/i
  ];

  for (const pattern of sqlPatterns) {
    if (pattern.test(q)) {
      defend.log('GCE-4451', 'sqli_blocked', {
        pattern: pattern.toString(),
        query: q.substring(0, 100),
        ip: req.ip,
        timestamp: Date.now()
      });
      return defend.block(400, {
        error: 'Invalid search query',
        code: 'GCE-4451-BLOCKED'
      });
    }
  }

  return req;
});
```

Strategy: Rewrite (Input Normalization)

```
// GCE-3387 · REWRITE strategy · Product API
// Vulnerability: Price manipulation via decimal precision
// Route: POST /api/cart/items

defend.intercept('POST /api/cart/items', (req) => {
  if (req.body.price !== undefined) {
    // Normalize: round to 2 decimal places, force positive
    const original = req.body.price;
    req.body.price = Math.abs(Math.round(original * 100) / 100);

    if (original !== req.body.price) {
      defend.log('GCE-3387', 'price_rewritten', {
        original,
        normalized: req.body.price,
        product_id: req.body.product_id,
        timestamp: Date.now()
      });
    }
  }

  return req;
});
```


Strategy: Header Inject (Security Headers)

```
// GCE-1024 · HEADER-INJECT strategy · All routes
// Vulnerability: Missing security headers on API responses

defend.intercept('*', (req, res, next) => {
  // Inject security headers on all responses
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('X-XSS-Protection', '1; mode=block');
  res.setHeader('Content-Security-Policy', "default-src 'self'");
  res.setHeader('Strict-Transport-Security', 'max-age=31536000; includeSubDomains');
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');

  defend.log('GCE-1024', 'headers_injected', {
    route: req.path,
    timestamp: Date.now()
  });

  return req;
});
```

Strategy: Redirect (Deprecated Endpoint)

```
// GCE-5561 · REDIRECT strategy · Legacy API
// Vulnerability: Deprecated endpoint with known vulnerability
// Route: POST /api/v1/legacy/auth

defend.intercept('POST /api/v1/legacy/auth', (req) => {
  defend.log('GCE-5561', 'deprecated_endpoint_redirected', {
    original: req.path,
    redirect: '/api/v2/auth',
    ip: req.ip,
    timestamp: Date.now()
  });

  // Redirect to secure v2 endpoint
  return defend.redirect(308, '/api/v2/auth');
});
```

4. SDK Integration Guide

The ShieldProbe SDK is deployed as a **middleware-layer library** inside your application. It is not a proxy, sidecar, or CDN rule — it hooks directly into the request processing pipeline of your framework.

4.1 Node.js SDK (Express/Fastify)

Installation

```
npm install @manticoreai/shieldprobe-sdk

# Environment variables
SHIELDPROBE_API_KEY=spk_live_xxxxxxx
SHIELDPROBE_ENV=production
SHIELDPROBE_GCE_ENABLED=true
```

Express Integration

```

const express = require('express');
const { ShieldProbe } = require('@manticoreai/shieldprobe-sdk');

const app = express();
app.use(express.json());

// Initialize ShieldProbe Defend SDK
const shield = new ShieldProbe({
  apiKey: process.env.SHIELDPROBE_API_KEY,
  environment: process.env.SHIELDPROBE_ENV || 'production',
  gceEnabled: true,           // Enable GCE runtime protection
  telemetryEnabled: true,    // Send latency/invoke metrics
  fallbackMode: 'report-only' // 'block' | 'report-only' | 'pass-through'
});

// Mount GCE middleware BEFORE route handlers
// This ensures GCE intercepts requests at the earliest point
app.use(shield.gceMiddleware());

// Health check (excluded from GCE processing)
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: Date.now() });
});

// Protected application routes
app.post('/api/withdraw', async (req, res) => {
  // At this point, GCE has already sanitized/validated the request
  const { account, amount } = req.body;

  // Business logic: GCE ensures amount >= 0
  const result = await processWithdrawal(account, amount);
  res.json(result);
});

app.post('/api/cart/items', async (req, res) => {
  // GCE has already normalized price to 2 decimal places
  const { product_id, price, quantity } = req.body;

  const item = await addToCart(product_id, price, quantity);
  res.json(item);
});

// Error handler for GCE blocks
app.use((err, req, res, next) => {
  if (err.code && err.code.startsWith('GCE-')) {
    // Log security event to your SIEM
    console.warn('GCE Security Event:', {
      code: err.code,
      route: req.path,
      ip: req.ip,
      timestamp: new Date().toISOString()
    });

    return res.status(err.status || 400).json({
      error: 'Request blocked by security policy',
      code: err.code,
      incident_id: err.incidentId
    });
  }
  next(err);
});

app.listen(3000, () => {

```

Fastify Integration

```
const fastify = require('fastify')({ logger: true });
const { ShieldProbe } = require('@manticoreai/shieldprobe-sdk/fastify');

// Register ShieldProbe as a Fastify plugin
fastify.register(ShieldProbe, {
  apiKey: process.env.SHIELDPROBE_API_KEY,
  gceEnabled: true,
  // Route-specific exclusions
  excludeRoutes: ['/health', '/metrics', '/_ready']
});

fastify.post('/api/transfer', async (request, reply) => {
  const { from, to, amount } = request.body;
  // GCE has validated and sanitized inputs
  return processTransfer(from, to, amount);
});

fastify.listen({ port: 3000 });
```

4.2 Python SDK (Django/Flask/FastAPI)

Installation

```
pip install manticore-shieldprobe

# Environment
export SHIELDPROBE_API_KEY="spk_live_xxxxxxx"
export SHIELDPROBE_ENV="production"
```

FastAPI Integration

```

from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import JSONResponse
from manticore_shieldprobe import ShieldProbeSDK

app = FastAPI()

# Initialize SDK
shield = ShieldProbeSDK(
    api_key="spk_live_XXXXXXX",
    environment="production",
    gce_enabled=True,
    fallback_mode="report-only"
)

# GCE middleware for FastAPI
@app.middleware("http")
async def shieldprobe_middleware(request: Request, call_next):
    # Skip health endpoints
    if request.url.path in ["/health", "/metrics"]:
        return await call_next(request)

    # GCE intercepts and processes request
    gce_result = await shield.process_request(request)

    if gce_result.blocked:
        await shield.log_event(gce_result)
        return JSONResponse(
            status_code=gce_result.status_code,
            content={
                "error": "Request blocked by security policy",
                "code": gce_result.gce_code,
                "incident_id": gce_result.incident_id
            }
        )

    # Request modified by GCE (sanitized/validated)
    if gce_result.modified:
        request = gce_result.request

    return await call_next(request)

@app.post("/api/withdraw")
async def withdraw(request: Request):
    body = await request.json()
    # amount is already sanitized by GCE
    result = await process_withdrawal(body["account"], body["amount"])
    return result

@app.on_event("startup")
async def startup():
    # Sync GCE rules from ManticoreAI platform
    await shield.sync_gces()
    print(f"ShieldProbe active with {shield.active_gce_count} GCE rules")

```

Django Middleware

```
# settings.py
MIDDLEWARE = [
    'manticore_shieldprobe.django.ShieldProbeMiddleware',
    'django.middleware.security.SecurityMiddleware',
    # ... other middleware
]

SHIELDPROBE = {
    'API_KEY': 'spk_live_xxxxxxx',
    'ENVIRONMENT': 'production',
    'GCE_ENABLED': True,
    'EXCLUDE_PATHS': ['/admin/', '/health/', '/static/'],
}

# The middleware automatically:
# 1. Syncs GCE rules on Django startup
# 2. Intercepts every request through process_request()
# 3. Applies GCE logic before view execution
# 4. Logs all security events
```

4.3 Go SDK (Gin/Echo/Standard)

```
package main

import (
    "log"
    "net/http"
    "github.com/gin-gonic/gin"
    shieldprobe "github.com/manticoreai/shieldprobe-sdk-go"
)

func main() {
    // Initialize ShieldProbe
    shield, err := shieldprobe.New(shieldprobe.Config{
        APIKey:      "spk_live_XXXXXXX",
        Environment: "production",
        GCEEnabled:  true,
        ExcludePaths: []string{"/health", "/metrics", "/_ready"},
    })
    if err != nil {
        log.Fatal(err)
    }

    // Sync GCE rules
    if err := shield.SyncGCEs(); err != nil {
        log.Printf("GCE sync warning: %v", err)
    }

    router := gin.New()

    // Use ShieldProbe middleware
    router.Use(shield.GinMiddleware())

    router.POST("/api/withdraw", func(c *gin.Context) {
        var req WithdrawRequest
        if err := c.ShouldBindJSON(&req); err != nil {
            c.JSON(400, gin.H{"error": err.Error()})
            return
        }

        // GCE has already sanitized req.Amount
        result, err := processWithdrawal(req.Account, req.Amount)
        if err != nil {
            c.JSON(500, gin.H{"error": err.Error()})
            return
        }
        c.JSON(200, result)
    })

    log.Fatal(router.Run(":8080"))
}

type WithdrawRequest struct {
    Account string `json:"account" binding:"required"`
    Amount  float64 `json:"amount" binding:"required"`
    // GCE ensures Amount >= 0 before this handler executes
}
```


4.4 Java SDK (Spring Boot)

```

// build.gradle dependency:
// implementation 'ai.manticore:shieldprobe-sdk-java:2.1.0'

// ShieldProbeConfig.java
@Configuration
public class ShieldProbeConfig {

    @Bean
    public ShieldProbeSDK shieldProbeSDK() {
        return ShieldProbeSDK.builder()
            .apiKey("spk_live_xxxxxxx")
            .environment("production")
            .gceEnabled(true)
            .fallbackMode(FallbackMode.REPORT_ONLY)
            .excludePaths(Arrays.asList("/health", "/actuator/**"))
            .build();
    }
}

// ShieldProbeFilter.java – Servlet Filter for GCE interception
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class ShieldProbeFilter extends OncePerRequestFilter {

    @Autowired
    private ShieldProbeSDK shield;

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain chain) throws ServletException, IOException {

        GCEResult result = shield.processRequest(request);

        if (result.isBlocked()) {
            response.setStatus(result.getStatusCode());
            response.setContentType("application/json");
            response.getWriter().write(String.format(
                "{\"error\": \"Request blocked\", \"code\": \"%s\", \"incident\": \"%s\"}",
                result.getGceCode(), result.getIncidentId()
            ));
            shield.logEvent(result);
            return;
        }

        // Continue with potentially modified request
        chain.doFilter(result.getRequest(), response);
    }
}

// WithdrawalController.java
@RestController
@RequestMapping("/api")
public class WithdrawalController {

    @PostMapping("/withdraw")
    public ResponseEntity<?> withdraw(@RequestBody WithdrawRequest req) {
        // GCE has sanitized amount to be >= 0
        BigDecimal safeAmount = req.getAmount().abs();
        WithdrawalResult result = withdrawalService.process(
            req.getAccount(), safeAmount
        );
        return ResponseEntity.ok(result);
    }
}

```


5. ShieldProbe Assess Specification

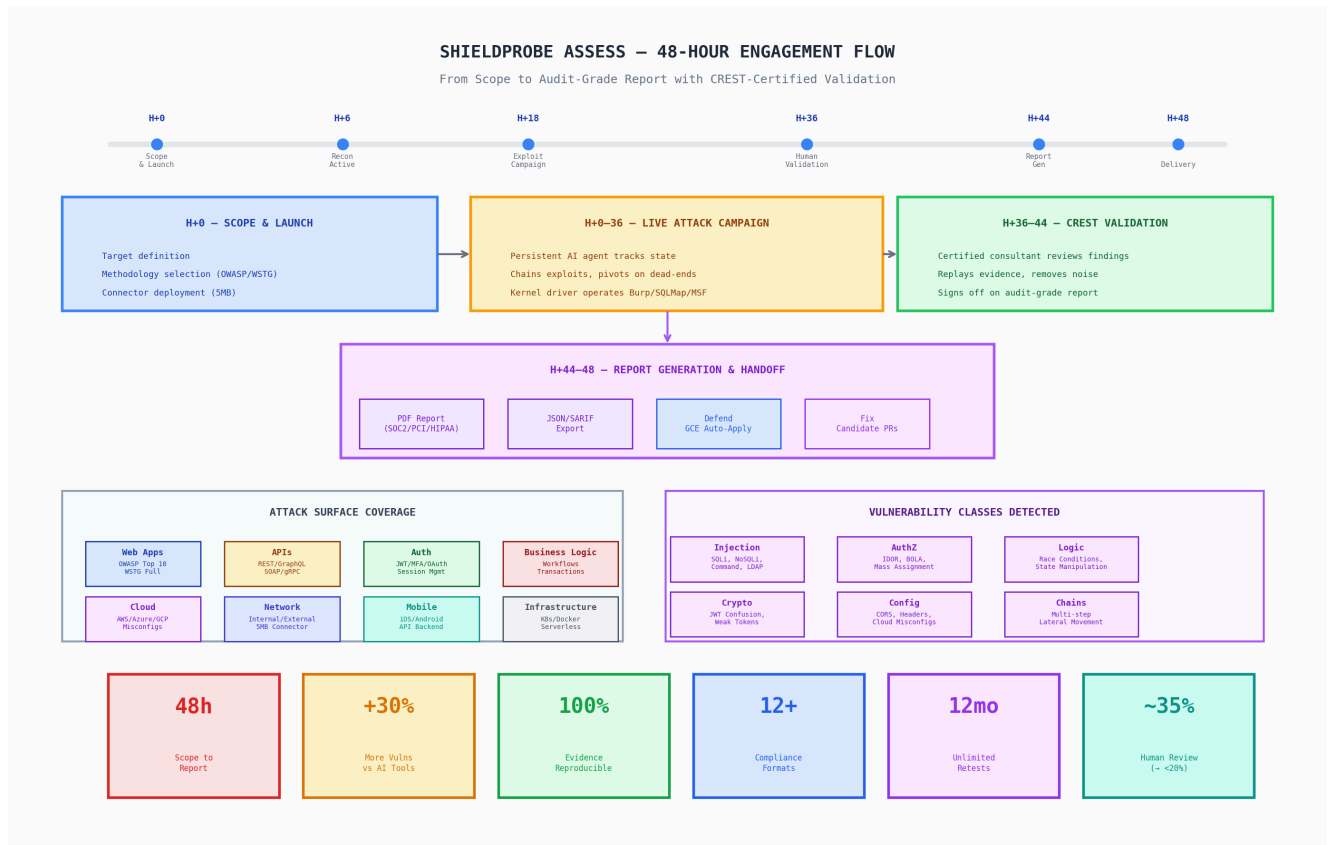


Figure 3 ShieldProbe Assess — 48-Hour Engagement Timeline and Attack Coverage

5.1 Engagement Execution Flow

Phase H+0: Scope and Launch

```
// Engagement launch API call
POST /api/v1/engagements/eng_9Kx2mP4vL7qR/launch

// Internal execution:
1. Validate target accessibility (TCP connect + HTTP probe)
2. Deploy SMB connector to internal network (if greybox)
3. Initialize attack state graph
4. Spawn persistent agent processes:
   - Recon agent: subdomain enumeration, tech fingerprinting
   - Auth agent: credential testing, session analysis
   - Logic agent: business flow mapping, state machine discovery
   - Exploit agent: payload generation, vulnerability validation
```

Phase H+0 to H+36: Live Attack Campaign

```
// Agent coordination loop (simplified pseudocode)
while engagement.active && time_elapsed < 36h:

    // Priority-based task scheduling
    tasks = prioritize_attack_surface(attack_graph)

    for task in tasks:
        // Run exploitation with full evidence capture
        result = execute_attack_task(task, {
            capture_requests: true,
            capture_responses: true,
            capture_screenshots: true,
            deterministic_replay: true
        })

        if result.vulnerability_confirmed:
            finding = create_finding({
                evidence: result.evidence,
                confidence: result.confidence,
                exploit_chain: result.chain,
                cwe_mapping: classify_cwe(result),
                cvss_score: calculate_cvss(result)
            })

            // Auto-trigger GCE generation for Defend
            if finding.severity in ['critical', 'high']:
                queue_gce_generation(finding)

            // Update attack graph with new intelligence
            attack_graph.update(result)

    // Every 6 hours: sync state, checkpoint progress
    if time_since_checkpoint > 6h:
        checkpoint_engagement()

    // Adaptive sleep based on coverage completeness
    sleep(calculate_backoff(attack_graph.coverage))
```

Phase H+36 to H+44: CREST Validation

```
// Validation pipeline
for finding in engagement.findings.where(status: 'pending'):

    // Step 1: Automated replay validation
    replay_result = replay_engine.execute(finding.replay_script)

    if replay_result.success:
        finding.add_validation('automated_replay', replay_result)
    else:
        finding.flag_for_human_review('replay_failed')
        continue

    // Step 2: RLHF quality scoring
    quality_score = rlhf_validator.score(finding)

    if quality_score < 0.85:
        finding.flag_for_human_review('low_quality_score')
        continue

    // Step 3: CREST consultant review (for critical/high)
    if finding.severity in ['critical', 'high']:
        crest_result = crest_queue.assign(finding)
        finding.sign_off(crest_result.consultant_id)
    else:
        finding.auto_approve()
```

5.2 Attack Surface Coverage Matrix

Table 11 Comprehensive Attack Surface Coverage

Category	Techniques	Tools Used	Evidence Type
Web Application	XSS, CSRF, XXE, SSRF, LFI/RFI, File Upload	Burp Suite, custom fuzzers	HTTP req/res, screenshots
API Security	IDOR, BOLA, Mass Assignment, Rate Limit Evasion	Burp, SQLMap, custom scripts	API req/res, auth tokens
Authentication	JWT Confusion, MFA Bypass, Session Fixation, OAuth	Custom frameworks	auth Token analysis, flow diagrams
Business Logic	Workflow Abuse, State Manipulation, Race Conditions	Deep-Process Engine	Context Full request chains
Injection	SQLi, NoSQLi, Command, LDAP, XPath	SQLMap, custom payloads	Payload + response proof
Cloud Config	S3 Buckets, IAM Roles, Network ACLs, Container Escapes	Prowler, ScoutSuite	CLI output, policy docs
Network	Port Scanning, Service Enumeration, SSL/TLS	Nmap, Metasploit	TestSSL, Scan results, cert analysis

5.3 Evidence Format Specification

Every finding includes a standardized evidence package:

```
{
  "evidence_format_version": "2.1.0",
  "finding_id": "fnd_3Xk9mP2vL5qR",
  "components": {
    "request": {
      "method": "POST",
      "url": "https://api.acme.com/api/withdraw",
      "headers": { "Authorization": "Bearer eyJhbG...", "Content-Type": "application/json" },
      "body": "{ \"account\": \"ACC-1234\", \"amount\": -50000 }",
      "timestamp": "2025-07-01T18:42:00Z"
    },
    "response": {
      "status_code": 200,
      "headers": { "Content-Type": "application/json" },
      "body": "{ \"status\": \"success\", \"new_balance\": 150000 }",
      "response_time_ms": 45
    },
    "screenshot": {
      "format": "png",
      "url": "https://cdn.manticore.ai/evidence/fnd_3Xk9.png",
      "dimensions": [1920, 1080],
      "annotated": true
    },
    "replay": {
      "deterministic": true,
      "script_url": "https://api.manticore.ai/v1/findings/fnd_3Xk9/replay",
      "script_format": "python_requests",
      "estimated_runtime_seconds": 3.5
    },
    "chain_context": [
      { "step": 1, "action": "API reconnaissance discovered /api/withdraw" },
      { "step": 2, "action": "Fuzzed amount parameter with negative values" },
      { "step": 3, "action": "Confirmed balance manipulation in response" }
    ]
  }
}
```

6. ShieldProbe Defend Specification

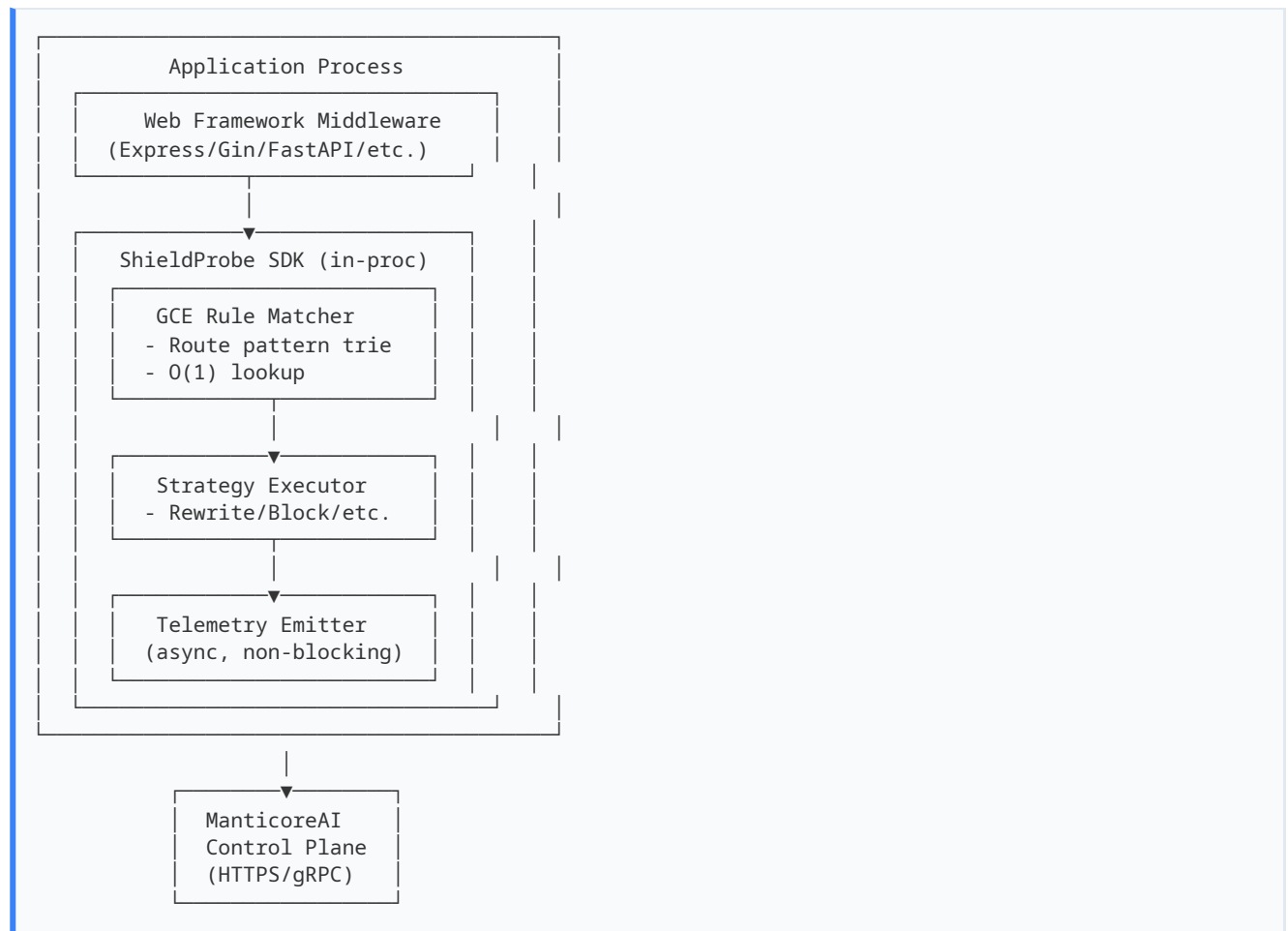
6.1 Deployment Architecture

Defend deploys as an **in-process SDK**, not a network proxy or sidecar. This architectural decision is fundamental to the 30ms latency guarantee.

Table 12 Deployment Model Comparison

Attribute	Traditional WAF	Sidecar Proxy	ShieldProbe Defend SDK
Network hop	Yes (DNS redirect)	Yes (localhost)	No (in-process)
Added latency	5–50ms	2–10ms	~27ms (total)
Context awareness	Limited (HTTP only)	Moderate	Full (app context)
Internal call coverage	No	No	Yes (function-to-function)
Deployment complexity	DNS change	K8s sidecar injection	Package install
Per-route precision	No (global rules)	URL-based	Yes (exact route patterns)

SDK Internal Architecture



6.2 Runtime Behavior Specification

GCE Rule Matching Algorithm

```
// 0(1) route matching via prefix trie
function matchGCE(request):
    routeKey = request.method + " " + request.path

    // Exact match first
    if gceTrie.hasExact(routeKey):
        return gceTrie.getExact(routeKey)

    // Parameterized route match
    if gceTrie.hasPattern(routeKey):
        return gceTrie.getPattern(routeKey)

    // Wildcard match (e.g., "POST /api/*")
    if gceTrie.hasWildcard(routeKey):
        return gceTrie.getWildcard(routeKey)

    // No GCE applicable – pass through
    return null

// Execution
function executeGCE(gce, request):
    startTime = now()

    try:
        result = gce.strategyExecutor.run(gce.code, request)
        latency = now() - startTime

        // Async telemetry (non-blocking)
        telemetry.emit({
            gce_id: gce.id,
            latency_ms: latency,
            strategy: gce.strategy,
            action: result.action, // 'pass', 'block', 'modify'
            timestamp: now()
        })

        return result
    catch error:
        // Fail-safe: log error, pass request through
        telemetry.emitError(gce.id, error)
        return { action: 'pass', reason: 'execution_error' }
```

Performance Characteristics

Table 13 Defend Runtime Performance

Metric	Value	Test Conditions
GCE lookup	<0.1ms	10,000 active GCE rules
Sanitize execution	0.5–2ms	JSON body, single field
Validate execution	0.3–1ms	Schema with 10 fields
Block execution	0.1–0.5ms	Pattern match on query string
Rewrite execution	0.5–3ms	Complex payload transformation
Memory overhead	~15MB	Per process, 1,000 GCEs loaded

GCE sync interval

30 seconds

Polling from control plane

Fail-Safe Behavior

Critical: Fail-Safe Design

If the ShieldProbe SDK encounters any error (network partition, malformed GCE code, control plane unreachable), it **always fails open** — requests pass through to the application unchanged. Security events during failures are logged locally and replayed when connectivity is restored. This design ensures GCE deployment never causes application downtime.

GCE Version Management

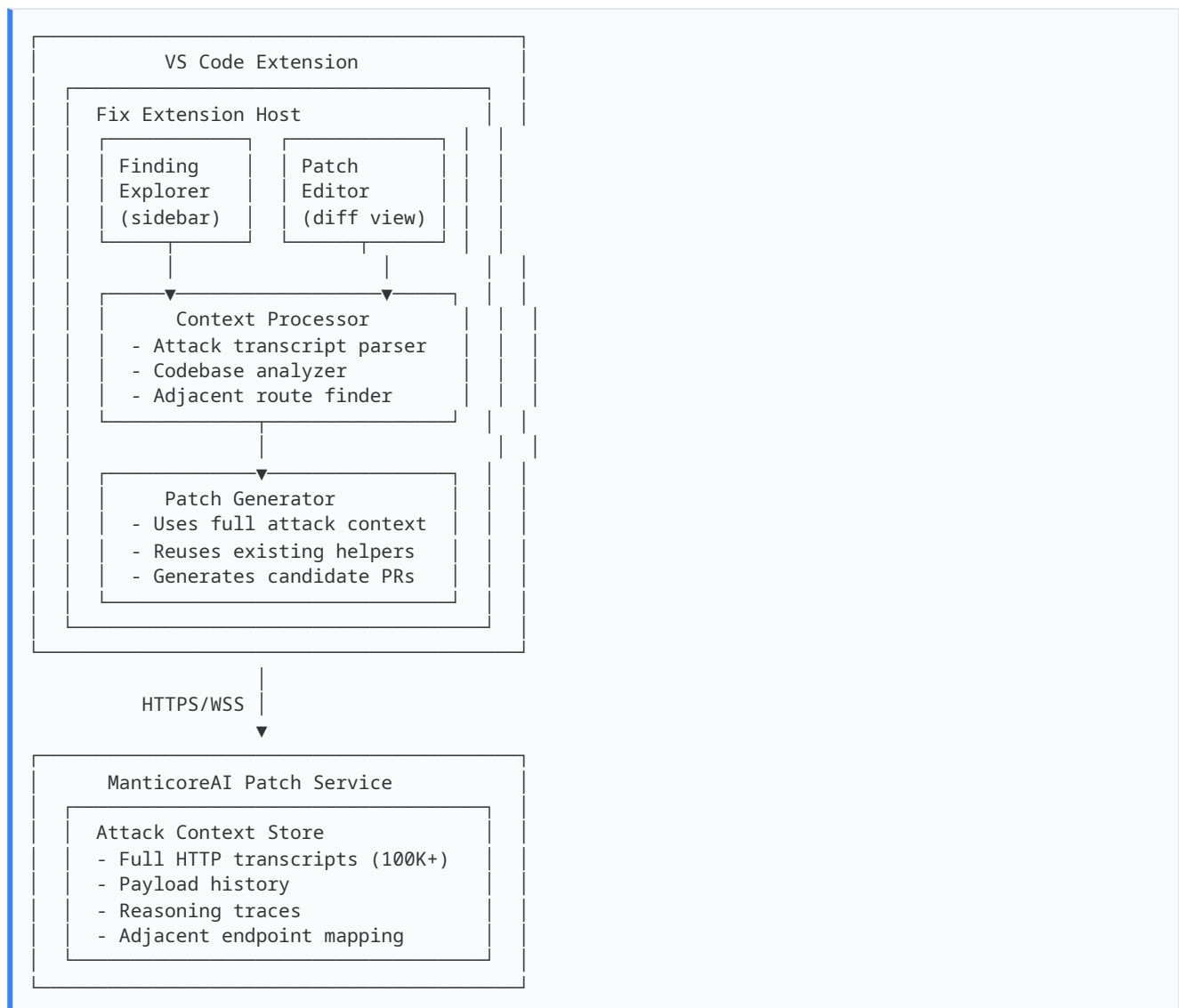
```
// Version rollback example (API call)
POST /api/v1/gces/gce_8Mk2pL5vX9qR/rollback
{
  "target_version": 2,
  "reason": "v3 causing false positives on valid refunds"
}

// SDK receives updated version within 30 seconds
// Old version code is preserved for audit trail
// Rollback is atomic – no request processed during switch
```

7. ShieldProbe Fix Specification

Fix is a VS Code extension that receives full pentest-agent context to generate higher-quality patches than generic AI coding assistants.

Technical Architecture



Patch Generation Algorithm

```
function generatePatch(finding, codebase):

    // Step 1: Extract attack context
    attackTranscript = fetchTranscript(finding.engagement_id)

    // Step 2: Analyze vulnerable code
    vulnerableFile = codebase.getFile(finding.location.file)
    vulnerableFunction = extractFunction(vulnerableFile, finding.location.line)

    // Step 3: Find adjacent routes sharing same pattern
    similarRoutes = codebase.findSimilarPatterns(vulnerableFunction)

    // Step 4: Discover existing helpers
    existingValidators = codebase.findFunctions('validate*')
    existingSanitizers = codebase.findFunctions('sanitize*')

    // Step 5: Generate context-aware patch
    patch = PatchGenerator.create({
        vulnerability: finding.category,
        strategy: selectFixStrategy(finding),
        originalCode: vulnerableFunction,
        adjacentRoutes: similarRoutes,
        existingHelpers: existingValidators + existingSanitizers,
        attackEvidence: attackTranscript,
        preserveApiContract: true // Don't change response shapes
    })

    // Step 6: Validate patch
    validation = PatchValidator.test(patch, {
        runUnitTests: true,
        checkSyntax: true,
        verifyNoRegression: true
    })

    return {
        diff: patch.diff,
        files_changed: patch.files,
        confidence: validation.score,
        tests_passed: validation.tests,
        pr_title: generatePRTitle(finding),
        pr_description: generatePRDescription(finding, patch)
    }
```

Safety Constraints (Hard-Coded)

Table 14 Fix Safety Constraints

Constraint	Implementation	Override Possible
Never auto-merge	All patches open as draft PRs only	No
Respect branch protection	Uses user's Git credentials, follows repo rules	No
No secrets generation	Hardcoded block on creds, tokens, keys	No
No infra changes	IaC files in deny-list	No
No third-party AI	Reasoning on ManticoreAI infra only	No

7. ShieldProbe Fix Specification

Max files per patch	Hard limit: 10 files	Admin-configurable
Min test coverage	Patch rejected if tests fail	Repo-configurable

8. ShieldProbe Continuous — CI/CD Integration

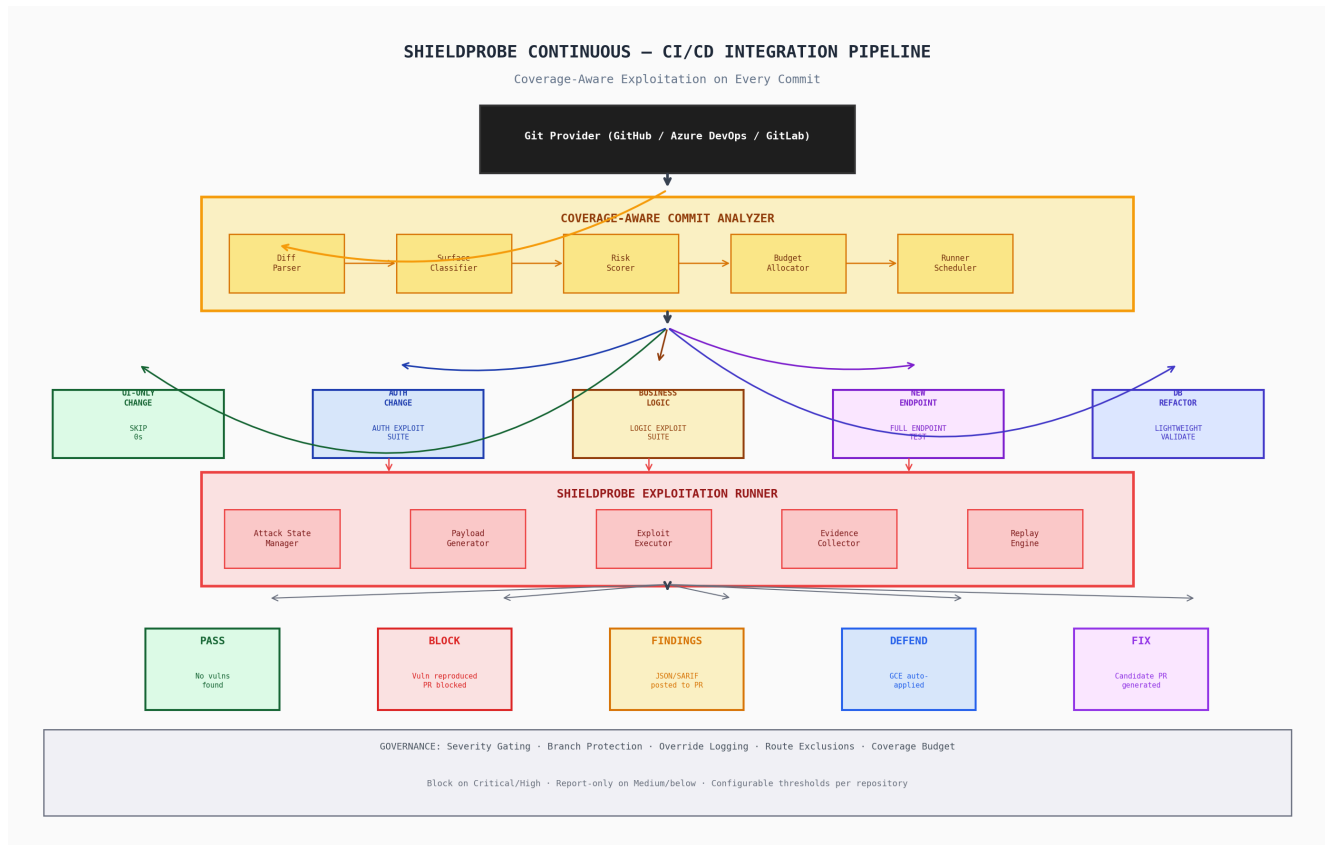


Figure 4 ShieldProbe Continuous — Coverage-Aware CI/CD Pipeline

Integration Modes

GitHub Actions

```
# .github/workflows/shieldprobe.yml
name: ShieldProbe Continuous Security

on:
  push:
    branches: [main, release/*]
  pull_request:
    branches: [main]

jobs:
  shieldprobe:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: ShieldProbe Continuous Scan
        uses: manticoreai/shieldprobe-action@v2
        with:
          api-key: ${ secrets.SHIELDPROBE_API_KEY }
          engagement-id: ${ vars.SHIELDPROBE_ENGAGEMENT_ID }
          coverage-budget: 120      # seconds per run
          severity-threshold: high  # block on high/critical
          fail-on-finding: true

      - name: Upload SARIF
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: shieldprobe-results.sarif
```

Azure DevOps Pipeline

```
# azure-pipelines.yml
trigger:
  branches:
    include: [main, release/*]

pool:
  vmImage: 'ubuntu-latest'

steps:
  - task: ShieldProbe@1
    inputs:
      apiKey: $(SHIELDPROBE_API_KEY)
      engagementId: $(SHIELDPROBE_ENGAGEMENT_ID)
      coverageBudget: 120
      severityThreshold: 'high'
      publishTestResults: true

  - task: PublishTestResults@2
    inputs:
      testResultsFormat: 'JUnit'
      testResultsFiles: 'shieldprobe-junit.xml'
```


Direct API Integration (Custom CI)

```
#!/usr/bin/env bash
# shieldprobe-ci.sh – Generic CI integration script

set -euo pipefail

API_KEY="${SHIELDPROBE_API_KEY}"
ENGAGEMENT_ID="${SHIELDPROBE_ENGAGEMENT_ID}"
COMMIT_HASH="$(git rev-parse HEAD)"
BRANCH="$(git branch --show-current)"
COVERAGE_BUDGET="${SHIELDPROBE_BUDGET:-120}"

# 1. Submit CI run
RUN_RESPONSE=$(curl -s -X POST "https://api.manticore.ai/api/v1/ci-runs" \
  -H "Authorization: Bearer ${API_KEY}" \
  -H "Content-Type: application/json" \
  -d "{
    \"engagement_id\": \"${ENGAGEMENT_ID}\",
    \"commit_hash\": \"${COMMIT_HASH}\",
    \"branch\": \"${BRANCH}\",
    \"provider\": \"custom\",
    \"coverage_budget_seconds\": ${COVERAGE_BUDGET}
  }")

RUN_ID=$(echo "$RUN_RESPONSE" | jq -r '.id')
echo "CI Run started: ${RUN_ID}"

# 2. Poll for completion
while true; do
  STATUS=$(curl -s "https://api.manticore.ai/api/v1/ci-runs/${RUN_ID}" \
    -H "Authorization: Bearer ${API_KEY}" | jq -r '.status')

  if [ "$STATUS" = "completed" ]; then
    break
  elif [ "$STATUS" = "failed" ]; then
    echo "ShieldProbe scan failed"
    exit 1
  fi

  echo "Status: ${STATUS}... waiting"
  sleep 10
done

# 3. Check results
RESULTS=$(curl -s "https://api.manticore.ai/api/v1/ci-runs/${RUN_ID}/findings" \
  -H "Authorization: Bearer ${API_KEY}")

CRITICAL=$(echo "$RESULTS" | jq '[.findings[] | select(.severity == "critical")] | length')
HIGH=$(echo "$RESULTS" | jq '[.findings[] | select(.severity == "high")] | length')

echo "Findings: ${CRITICAL} critical, ${HIGH} high"

if [ "$CRITICAL" -gt 0 ] || [ "$HIGH" -gt 0 ]; then
  echo "Failing build due to security findings"
  exit 1
fi

echo "Security check passed"
```

Coverage-Aware Analysis Rules

Table 15 Commit Classification Rules

Change Type	Detection	Action	Typical Duration
UI-only (CSS/HTML)	File extension + path pattern	Skip (0s)	0 seconds
Documentation	*.md, *.txt, /docs/	Skip	0 seconds
Auth module change	auth*, login*, session*, jwt*	Auth exploit suite	30–120s
Payment/billing	payment*, billing*, stripe*, checkout*	Business logic suite	60–240s
New API endpoint	Added route definitions	Full endpoint test	30–180s
Database change	migration*, schema*, *.sql	Data validation	15–60s
Dependency update	package.json, requirements.txt	CVE scan + targeted test	20–90s
Configuration	*.yaml, *.yml, *.env*	Config security scan	10–30s

9. Database Schema

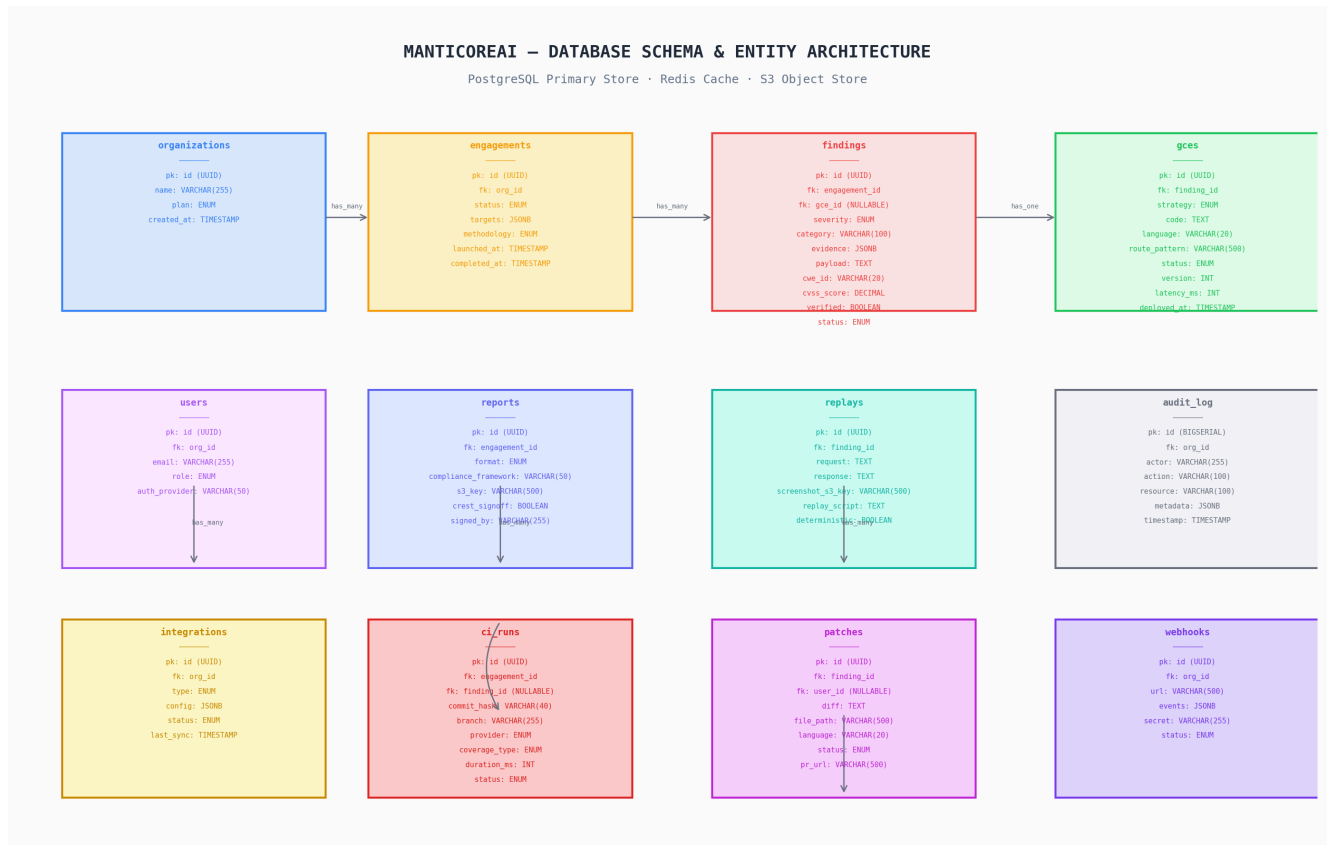


Figure 5 PostgreSQL Primary Store — Entity Relationships and Schema Definitions

Core Table Definitions

organizations

```

CREATE TABLE organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(100) UNIQUE NOT NULL,
  plan VARCHAR(50) NOT NULL DEFAULT 'starter'
  CHECK (plan IN ('starter', 'professional', 'enterprise')),
  settings JSONB NOT NULL DEFAULT '{}',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_orgs_slug ON organizations(slug);
CREATE INDEX idx_orgs_plan ON organizations(plan);

```

engagements

```
CREATE TABLE engagements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  status VARCHAR(50) NOT NULL DEFAULT 'scheduled'
    CHECK (status IN ('scheduled', 'running', 'validating', 'completed', 'cancelled')),
  targets JSONB NOT NULL DEFAULT '[]',
  methodology VARCHAR(50) NOT NULL DEFAULT 'owasp_top10'
    CHECK (methodology IN ('owasp_top10', 'wstg_full', 'pci_dss', 'custom')),
  testing_type VARCHAR(20) NOT NULL DEFAULT 'greybox'
    CHECK (testing_type IN ('blackbox', 'greybox', 'whitebox')),
  compliance_frameworks JSONB NOT NULL DEFAULT '[]',
  findings_summary JSONB,
  launched_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_engagements_org ON engagements(org_id);
CREATE INDEX idx_engagements_status ON engagements(status);
CREATE INDEX idx_engagements_dates ON engagements(launched_at, completed_at);
```

findings

```
CREATE TABLE findings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  engagement_id UUID NOT NULL REFERENCES engagements(id) ON DELETE CASCADE,
  gce_id UUID,
  severity VARCHAR(20) NOT NULL
    CHECK (severity IN ('critical', 'high', 'medium', 'low', 'info')),
  category VARCHAR(100) NOT NULL,
  title VARCHAR(500) NOT NULL,
  description TEXT NOT NULL,
  evidence JSONB NOT NULL DEFAULT '{}',
  payload TEXT,
  cwe_id VARCHAR(20),
  cvss_score DECIMAL(3,1),
  verified BOOLEAN NOT NULL DEFAULT false,
  verified_by VARCHAR(255),
  status VARCHAR(50) NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'verified', 'false_positive', 'remediated', 'accepted_risk')),
  attack_chain JSONB,
  affected_routes JSONB NOT NULL DEFAULT '[]',
  remediation TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  verified_at TIMESTAMPTZ
);

CREATE INDEX idx_findings_engagement ON findings(engagement_id);
CREATE INDEX idx_findings_severity ON findings(severity);
CREATE INDEX idx_findings_status ON findings(status);
CREATE INDEX idx_findings_gce ON findings(gce_id) WHERE gce_id IS NOT NULL;
CREATE INDEX idx_findings_category ON findings(category);
-- Full-text search on title and description
CREATE INDEX idx_findings_search ON findings USING gin(to_tsvector('english', title || ' ' ||
  coalesce(description, '')));
```

gces (Generative Counter Exploits)

```

CREATE TABLE gces (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  finding_id UUID NOT NULL REFERENCES findings(id) ON DELETE CASCADE,
  strategy VARCHAR(50) NOT NULL
    CHECK (strategy IN ('rewrite', 'block', 'sanitize', 'validate', 'redirect',
'header_inject')),
  code TEXT NOT NULL,
  language VARCHAR(20) NOT NULL,
  route_pattern VARCHAR(500) NOT NULL,
  status VARCHAR(50) NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'active', 'disabled', 'deprecated', 'rolled_back')),
  version INT NOT NULL DEFAULT 1,
  latency_ms INT,
  invocations BIGINT NOT NULL DEFAULT 0,
  threats_blocked BIGINT NOT NULL DEFAULT 0,
  deployed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_gces_finding ON gces(finding_id);
CREATE INDEX idx_gces_status ON gces(status);
CREATE INDEX idx_gces_strategy ON gces(strategy);

```

ci_runs

```

CREATE TABLE ci_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  engagement_id UUID NOT NULL REFERENCES engagements(id) ON DELETE CASCADE,
  finding_id UUID REFERENCES findings(id),
  commit_hash VARCHAR(40) NOT NULL,
  branch VARCHAR(255) NOT NULL,
  provider VARCHAR(50) NOT NULL
    CHECK (provider IN ('github', 'azure_devops', 'gitlab', 'jenkins', 'custom')),
  coverage_type VARCHAR(50),
  coverage_budget_seconds INT NOT NULL DEFAULT 120,
  duration_ms INT,
  findings_count INT NOT NULL DEFAULT 0,
  status VARCHAR(50) NOT NULL DEFAULT 'running'
    CHECK (status IN ('queued', 'running', 'completed', 'failed', 'skipped')),
  results JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ
);

CREATE INDEX idx_ci_runs_engagement ON ci_runs(engagement_id);
CREATE INDEX idx_ci_runs_commit ON ci_runs(commit_hash);
CREATE INDEX idx_ci_runs_status ON ci_runs(status);

```

Redis Key Structure

```
// Session and caching layer
shieldprobe:session:{session_id}    -> JWT payload (TTL: 24h)
shieldprobe:org:{org_id}:gces        -> Active GCE rule set (TTL: 5min)
shieldprobe:rate:{client_ip}         -> Rate limit counter (TTL: 1min)
shieldprobe:gce:{gce_id}:metrics     -> Runtime metrics (TTL: 24h)
shieldprobe:eng:{eng_id}:status      -> Engagement status cache (TTL: 30s)
shieldprobe:replay:{finding_id}      -> Deterministic replay data (TTL: 90d)
```

Kafka Topic Schema

Table 16 Kafka Event Topics

Topic	Partitioning	Retention	Schema
findings.discovered	engagement_id (12 partitions)	90 days	Avro
gce.deployments	org_id (6 partitions)	30 days	Avro
telemetry.metrics	sdk_instance (24 partitions)	7 days	Protobuf
audit.events	org_id (6 partitions)	1 year	Avro
ci.runs	repo_id (12 partitions)	30 days	Avro

10. Authentication & Security Model

Multi-Factor Authentication

Table 17 Supported Authentication Methods

Method	Type	Status
Username + Password	Primary	Supported
Time-based OTP (TOTP)	Secondary	Supported (Google Authenticator, Authy)
WebAuthn / FIDO2	Secondary	Supported (YubiKey, Touch ID)
SAML 2.0	Federated	Enterprise tier
OIDC (Google, Azure AD)	Federated	All tiers
SCIM Provisioning	Directory sync	Enterprise tier

Authorization Model (RBAC)

```
// Role definitions
const ROLES = {
  owner: {
    permissions: ['*'], // Full access
    description: 'Organization owner'
  },
  admin: {
    permissions: [
      'engagements:*', 'findings:*', 'gces:*',
      'reports:*', 'integrations:*', 'team:manage',
      'settings:read', 'settings:write', 'billing:*'
    ]
  },
  security_engineer: {
    permissions: [
      'engagements:create', 'engagements:read',
      'findings:*', 'gces:manage',
      'reports:create', 'reports:read',
      'integrations:read', 'integrations:write'
    ]
  },
  developer: {
    permissions: [
      'engagements:read',
      'findings:read', 'findings:update_status',
      'gces:read', 'reports:read',
      'fix:use' // Can use IDE extension
    ]
  },
  auditor: {
    permissions: [
      'engagements:read', 'findings:read',
      'reports:read', 'audit_log:read'
    ]
  },
  viewer: {
    permissions: [
      'engagements:read', 'findings:read',
      'reports:read'
    ]
  }
}
};
```


API Key Scoping

```
// Scoped API keys with fine-grained permissions
{
  "key_id": "spk_live_2vX9kL3mP7qR",
  "name": "CI/CD Integration Key",
  "scopes": [
    "assess:read",
    "findings:read",
    "ci_runs:create",
    "ci_runs:read"
  ],
  "restrictions": {
    "allowed_ips": ["192.168.1.0/24", "10.0.0.0/8"],
    "allowed_repos": ["acme/web-app", "acme/api-service"],
    "expires_at": "2025-12-31T23:59:59Z"
  }
}
```

Data Encryption

Table 18 Encryption Specifications

Layer	Method	Key Management
Data at rest (DB)	AES-256-GCM	HashiCorp Vault with auto-rotation
Data at rest (S3)	AES-256 (server-side)	AWS KMS / Cloudflare R2
Data in transit	TLS 1.3 (mandatory)	Let's Encrypt / Cloudflare
Inter-service	mTLS (SPKI)	Vault PKI engine
Secrets	AES-256-GCM + envelope	Vault transit engine
API keys	bcrypt hash (display: prefix only)	Hashed in PostgreSQL

11. Webhook/Event Reference

Webhook Delivery Format

```
POST /your-webhook-endpoint HTTP/1.1
Host: your-domain.com
X-Manticore-Signature: sha256=a6f5c8d2e4b1...
X-Manticore-Event: finding.discovered
X-Manticore-Delivery: dlv_9Kx2mP4vL7qR
Content-Type: application/json

{
  "event": "finding.discovered",
  "delivery_id": "dlv_9Kx2mP4vL7qR",
  "timestamp": "2025-07-01T18:42:00Z",
  "data": {
    "finding_id": "fnd_3Xk9mP2vL5qR",
    "engagement_id": "eng_9Kx2mP4vL7qR",
    "severity": "critical",
    "category": "business_logic",
    "title": "Negative-amount withdrawal balance inflation",
    "cvss_score": 8.7,
    "affected_route": "POST /api/withdraw",
    "dashboard_url": "https://app.manticore.ai/findings/fnd_3Xk9mP2vL5qR"
  }
}
```

Webhook Signature Verification

```
const crypto = require('crypto');

function verifyWebhook(payload, signature, secret) {
  const expected = crypto
    .createHmac('sha256', secret)
    .update(payload, 'utf8')
    .digest('hex');

  const actual = signature.replace('sha256=', '');

  return crypto.timingSafeEqual(
    Buffer.from(expected, 'hex'),
    Buffer.from(actual, 'hex')
  );
}

// Usage
app.post('/webhooks/manticore', express.raw({type: 'application/json'}), (req, res) => {
  const signature = req.headers['x-manticore-signature'];
  const secret = process.env.MANTICORE_WEBHOOK_SECRET;

  if (!verifyWebhook(req.body, signature, secret)) {
    return res.status(401).send('Invalid signature');
  }

  const event = JSON.parse(req.body);

  switch (event.event) {
    case 'finding.discovered':
      // Notify team via Slack/PagerDuty
      await notifySecurityTeam(event.data);
      break;
    case 'engagement.completed':
      // Auto-schedule remediation sprint
      await createRemediationTickets(event.data);
      break;
    case 'gce.triggered':
      // Log to SIEM
      await siem.logSecurityEvent(event.data);
      break;
  }

  res.status(200).send('OK');
});
```

12. Working Integration Examples

Example 1: Full CI/CD with Auto-Remediation

```
// Complete GitHub Actions workflow with ShieldProbe
// .github/workflows/security.yml

name: Full Security Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 2 * * 1' // Weekly Monday 2 AM

jobs:
  shieldprobe-scan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
      security-events: write
    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0 // Full history for coverage analysis

      - name: ShieldProbe Continuous
        id: shieldprobe
        uses: manticoreai/shieldprobe-action@v2
        with:
          api-key: ${ secrets.SHIELPROBE_API_KEY }
          engagement-id: ${ vars.SHIELPROBE_ENGAGEMENT_ID }
          coverage-budget: 180
          severity-threshold: medium

      - name: Upload SARIF to GitHub Security Tab
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: shieldprobe-results.sarif

      - name: Comment PR with findings
        if: github.event_name == 'pull_request' && steps.shieldprobe.outputs.findings_count > 0
        uses: actions/github-script@v7
        with:
          script: |
            const findings = JSON.parse('${ steps.shieldprobe.outputs.findings }');
            const body = findings.map(f =>
              '- **${f.severity.toUpperCase()}**: ${f.title} (${f.category})`
            ).join('\n');

            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: `## ShieldProbe Security Findings\n\n${body}\n\n[View Full Report](${ steps.shieldprobe.outputs.report_url })`
            });

      - name: Deploy GCEs (main branch only)
        if: github.ref == 'refs/heads/main'
        uses: manticoreai/shieldprobe-deploy-action@v1
        with:
          api-key: ${ secrets.SHIELPROBE_API_KEY }
          auto-enable: true
```

Example 2: Custom SIEM Integration (Splunk)

```

// shieldprobe-splunk-connector.js
const { SplunkSender } = require('./splunk');
const shield = require('@manticoreai/shieldprobe-sdk');

class ShieldProbeSplunkConnector {
  constructor(config) {
    this.splunk = new SplunkSender(config.splunk);
    this.shield = new shield.WebhookServer({
      port: config.port || 3000,
      secret: config.webhookSecret
    });
  }

  async start() {
    // Handle GCE trigger events
    this.shield.on('gce.triggered', async (event) => {
      await this.splunk.send({
        sourcetype: 'manticore:gce',
        event: {
          action: 'gce_triggered',
          gce_id: event.data.gce_id,
          finding_id: event.data.finding_id,
          route: event.data.route_pattern,
          strategy: event.data.strategy,
          client_ip: event.data.client_ip,
          blocked_at: event.timestamp,
          severity: 'info'
        }
      });
    });

    // Handle new findings
    this.shield.on('finding.discovered', async (event) => {
      await this.splunk.send({
        sourcetype: 'manticore:finding',
        event: {
          action: 'finding_discovered',
          finding_id: event.data.finding_id,
          severity: event.data.severity,
          category: event.data.category,
          title: event.data.title,
          cvss_score: event.data.cvss_score,
          dashboard_url: event.data.dashboard_url,
          severity_level: this.mapSeverity(event.data.severity)
        }
      });

      // Auto-create PagerDuty incident for critical
      if (event.data.severity === 'critical') {
        await this.createPagerDutyIncident(event.data);
      }
    });

    await this.shield.start();
    console.log('ShieldProbe - Splunk connector running');
  }

  mapSeverity(severity) {
    const map = { critical: 6, high: 5, medium: 4, low: 3, info: 2 };
    return map[severity] || 1;
  }
}

// Start
new ShieldProbeSplunkConnector({

```


Example 3: Terraform Deployment

```
// main.tf – ManticoreAI integration infrastructure

# ShieldProbe webhook receiver
resource "aws_apigatewayv2_api" "shieldprobe_webhook" {
  name          = "shieldprobe-webhook"
  protocol_type = "HTTP"
}

resource "aws_lambda_function" "shieldprobe_handler" {
  filename      = "shieldprobe-handler.zip"
  function_name = "shieldprobe-webhook-processor"
  role          = aws_iam_role.lambda_role.arn
  handler       = "index.handler"
  runtime       = "nodejs20.x"

  environment {
    variables = {
      SHIELDPROBE_SECRET = var.shieldprobe_secret
      SLACK_WEBHOOK_URL   = var.slack_webhook
      JIRA_API_TOKEN      = var.jira_token
    }
  }
}

# Security group for ShieldProbe connector (internal testing)
resource "aws_security_group" "shieldprobe_connector" {
  name        = "shieldprobe-connector"
  description = "Egress-only security group for ShieldProbe pentest connector"

  egress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTPS to ManticoreAI API"
  }

  egress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["10.0.0.0/8"]
    description = "Internal web app testing"
  }
}

# WAF rule for defense-in-depth (complements GCE)
resource "aws_wafv2_web_acl" "shieldprobe_layer" {
  name        = "shieldprobe-defense-in-depth"
  scope       = "REGIONAL"

  rule {
    name      = "RateLimit"
    priority  = 1

    action {
      block {}
    }

    statement {
      rate_based_statement {
        limit              = 2000
        aggregate_key_type = "IP"
      }
    }
  }
}
```

Example 4: Jira Ticketing Automation

```
// shieldprobe-jira.js – Auto-create tickets from findings

const JiraClient = require('jira-client');

class ShieldProbeJiraIntegration {
  constructor() {
    this.jira = new JiraClient({
      protocol: 'https',
      host: process.env.JIRA_HOST,
      username: process.env.JIRA_USER,
      password: process.env.JIRA_TOKEN,
      apiVersion: '2'
    });
  }

  async handleFindingDiscovered(event) {
    const finding = event.data;

    // Map severity to Jira priority
    const priorityMap = {
      critical: '1', // Highest
      high: '2', // High
      medium: '3', // Medium
      low: '4', // Low
      info: '5' // Lowest
    };

    // Map category to Jira component
    const componentMap = {
      business_logic: 'Business Logic',
      injection: 'Injection Vulnerability',
      authentication: 'Authentication',
      authorization: 'Authorization',
      configuration: 'Configuration',
      cryptography: 'Cryptography'
    };

    const issue = await this.jira.addNewIssue({
      fields: {
        project: { key: process.env.JIRA_PROJECT_KEY },
        summary: `[${finding.severity.toUpperCase()}] ${finding.title}`,
        description: this.formatDescription(finding),
        issuetype: { name: 'Security Vulnerability' },
        priority: { id: priorityMap[finding.severity] },
        components: [{ name: componentMap[finding.category] || 'General' }],
        labels: ['shieldprobe', finding.severity, finding.category],
        customfield_10001: finding.cvss_score, // CVSS Score
        customfield_10002: finding.cwe_id // CWE ID
      }
    });

    // Link back to ManticoreAI dashboard
    await this.jira.addComment(issue.key,
      `*ManticoreAI Finding:* ${finding.dashboard_url}\n` +
      `*Auto-created by ShieldProbe integration*`
    );

    console.log(`Created Jira issue ${issue.key} for finding ${finding.finding_id}`);
    return issue;
  }

  formatDescription(finding) {
    return `h2. Security Finding Details

*Severity:* ${finding.severity.toUpperCase()}`
  }
}
```


Appendix A: HTTP Status Codes

Table 19 API Response Status Codes

Code	Meaning	Typical Use
200 OK	Success	GET requests, successful operations
201 Created	Resource created	POST /engagements, POST /reports
202 Accepted	Async operation started	Report generation, CI run
204 No Content	Success, no body	DELETE, PATCH on status
400 Bad Request	Validation error	Invalid request body
401 Unauthorized	Auth required	Missing/invalid token
403 Forbidden	Insufficient permissions	Role-based access denied
404 Not Found	Resource not found	Invalid engagement/finding ID
409 Conflict	Resource conflict	Duplicate engagement name
422 Unprocessable	Business rule violation	Invalid target URL
429 Too Many Requests	Rate limit exceeded	API throttling
500 Server Error	Internal error	Unexpected failure
503 Service Unavailable	Maintenance/overload	Scheduled maintenance

Appendix B: Rate Limits

Table 20 API Rate Limit Tiers

Tier	Requests/Minute	Burst	CI Runs/Day
Starter	60	10	50
Professional	300	50	200
Enterprise	1,000	200	Unlimited

Appendix C: SDK Support Matrix

Table 21 Language and Framework Support

Language	Frameworks	Min Version	GCE	Telemetry
----------	------------	-------------	-----	-----------

Node.js	Express, Fastify, Koa, Hapi	18.x	Yes	Full
Python	FastAPI, Django, Flask	3.10	Yes	Full
Go	Gin, Echo, stdlib	1.21	Yes	Full
Java	Spring Boot, Jakarta EE	17	Yes	Full
Ruby	Rails, Sinatra	3.2	Yes	Basic
.NET	ASP.NET Core	8.0	Yes	Full
PHP	Laravel, Symfony	8.2	Yes	Basic
Rust	Actix, Axum	1.75	Preview	Basic

Document Version

This technical specification reflects the ManticoreAI ShieldProbe platform as of June 2026. API schemas, SDK versions, and integration endpoints are subject to change. For the latest reference, consult <https://docs.manticore.ai> or the OpenAPI specification at <https://api.manticore.ai/openapi.json>.